

p1059R0

Adapting Asio to use std::expected

Published Proposal, 6 May 2018

Author:

[Thomas Rodgers](#) (RedHat)

Audience:

LEWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

An experience report on adapting an existing API to use expected

Table of Contents

- 1 **Background**
- 2 **Two signatures for most operations**
- 3 **The Problem with Partial Failure**
- 4 **Asynchronous Completion Handlers**
- 5 **Adapting User Code**

References

Informative References

§ 1. Background

This paper explores adapting the API of the Asio networking library to use the expected type proposed in [\[p0323\]](#).

Asio's [\[asio\]](#) API, like that of the `<filesystem>` and the Networking TS [\[n4711\]](#), use pairs of functions for each operation, one which is expected to report disappointment's via exceptions, and one which will return underlying API disappointments via `error_code` (but may also throw, e.g. `bad_alloc`).

§ 2. Two signatures for most operations

Asio [\[asio\]](#), like and the proposed Networking TS [\[n4711\]](#) declare variants of most entry points which either report all errors as exceptions, or some, typically low level system API results, as an `error_code` passed to the call by reference. For example -

```
// This form throws on error
template <typename SyncReadStream, typename MutableBufferSequence>
std::size_t read(SyncReadStream& s, const MutableBufferSequence& buffers,
    typename enable_if<
        is_mutable_buffer_sequence<MutableBufferSequence>::value
        >::type* = 0);

// This form returns error, if any in ec
template <typename SyncReadStream, typename MutableBufferSequence>
std::size_t read(SyncReadStream& s, const MutableBufferSequence& buffers,
    asio::error_code& ec);
```

Note, the second form is not marked `noexcept`, and thereform may also report errors by throwing, e.g. `std::bad_alloc`.

It appears that the second form should be the one adapted for use with `expected<T,E>`. One possibility is -

```
using expected_result = std::expected<std::size_t, asio::error_code>;
template <typename SyncReadStream, typename MutableBufferSequence>
expected_result read(SyncReadStream& s, const MutableBufferSequence& buffer
```

Another possibility is to promote `error_code` to `error_status` as well as making the signature of the function `noexcept` -

```
using expected_result = std::expected<std::size_t, std::exception>;
template <typename SyncReadStream, typename MutableBufferSequence>
expected_result read(SyncReadStream& s, const MutableBufferSequence& buffer
```

Both of these approaches suffer from a common problem, namely the call signature is a match for the original throwing version of `read`. One possible approach is to use tag dispatching to resolve the ambiguity -

```
using expected_result = std::expected<std::size_t, asio::error_code>;
template <typename SyncReadStream, typename MutableBufferSequence>
expected_result read(SyncReadStream& s, const MutableBufferSequence& buffer,
                    std::nothrow_t);
```

This approach suffers from the problem that the tag is misleading, because this function can in fact throw. This argues perhaps then for promoting `error_code` to an exception.

```
using expected_result = std::expected<std::size_t, std::exception>;
template <typename SyncReadStream, typename MutableBufferSequence>
expected_result read(SyncReadStream& s, const MutableBufferSequence& buffer,
                    std::nothrow_t) noexcept;
```

§ 3. The Problem with Partial Failure

Asio's "composed" operations, in the form of freestanding functions such as `asio::read()` and `asio::write()` are implemented in terms of zero or more calls to the supplied stream's `stm.read_more()` and `stm.write_more()` methods. While an individual call to the underlying stream type's methods might represent a single failed call, successful calls to the underlying stream may likely have occurred, resulting some amount of data being placed into a buffer by `stm.read_some()` or written to the stream by `stm.write_some()`.

As such, a valid outcome for these operations is a failed status, for an individual underlying call placed into the supplied `error_code& ec` parameter, and the number of bytes transferred thus far. Naively converting the `error_code` accepting functions to returning `expected<std::size_t,`

`asio::error_code` will result in the loss of the bytes transferred result which breaks legitimate uses of these functions in the face partial, recoverable errors.

Note, The throwing versions of these "composed" functions also suffer from this issue

This suggests that perhaps there is a related type representing a partial failure, that can decay to `std::expected`.

§ 4. Asynchronous Completion Handlers

As is the case with the Networking TS, Asio defines asynchronous versions of many operations. A typical completion handler conforms to the following concept -

```
void handler(const asio::error_code& error, std::size_t bytes_transferred);
```

Which is then supplied to an async request function -

```
template <typename AsyncReadStream, typename MutableBufferSequence,
          typename ReadHandler>
void async_read(AsyncReadStream& s, const MutableBufferSequence& buffers,
               ReadHandler&& handler);
```

It is simple enough to introduce compile time dispatching to allow `async_read` to dispatch to a completion handler taking an expected -

```
using handler_result = std::expected<std::size_t, asio::error_code>;
void handler(handler_result r);
```

The impact to the user here is fairly minimal in the simple case, however it allows the possibility that the user could supply a handler of the form -

```
class my_handler {
    void operator()(const asio::error_code& error, std::size_t bytes_transferred) { ... }
    void operator()(std::expected<std::size_t, asio::error_code> r) { ... }
};
```

Should priority be given to one signature vs the other?

§ 5. Adapting User Code

Asio includes the following example of a simple blocking client -

```

int main(int argc, char* argv[])
{
    try
    {
        if (argc != 3)
        {
            std::cerr << "Usage: blocking_udp_echo_client <host> <port>\n";
            return 1;
        }

        asio::io_context io_context;

        udp::socket s(io_context, udp::endpoint(udp::v4(), 0));

        udp::resolver resolver(io_context);
        udp::resolver::results_type endpoints =
            resolver.resolve(udp::v4(), argv[1], argv[2]);

        std::cout << "Enter message: ";
        char request[max_length];
        std::cin.getline(request, max_length);
        size_t request_length = std::strlen(request);
        s.send_to(asio::buffer(request, request_length), *endpoints.begin());

        char reply[max_length];
        udp::endpoint sender_endpoint;
        size_t reply_length = s.receive_from(
            asio::buffer(reply, max_length), sender_endpoint);
        std::cout << "Reply is: ";
        std::cout.write(reply, reply_length);
        std::cout << "\n";
    }
    catch (std::exception& e)
    {
        std::cerr << "Exception: " << e.what() << "\n";
    }

    return 0;
}

```

This client relies on the throwing versions of the Asio API functions. A version that relied instead on handling `error_code` might read -

```

int main(int argc, char* argv[])
{
    if (argc != 3)
    {
        std::cerr << "Usage: blocking_udp_echo_client <host> <port>\n";
        return 1;
    }

    asio::io_context io_context;

    udp::socket s(io_context, udp::endpoint(udp::v4(), 0));

    error_code ec;
    udp::resolver resolver(io_context);
    udp::resolver::results_type endpoints =
        resolver.resolve(udp::v4(), argv[1], argv[2], ec);

    if (ec)
        return report(ec);

    std::cout << "Enter message: ";
    char request[max_length];
    std::cin.getline(request, max_length);
    size_t request_length = std::strlen(request);

    s.send_to(asio::buffer(request, request_length), *endpoints.begin(), ec);
    if (ec)
        return report(ec);

    char reply[max_length];
    udp::endpoint sender_endpoint;
    size_t reply_length = s.receive_from(
        asio::buffer(reply, max_length), sender_endpoint, ec);

    if (ec)
        return report(ec);

    std::cout << "Reply is: ";
    std::cout.write(reply, reply_length);
    std::cout << "\n";
}

```

```
return 0;  
}
```

This same client, but using `expected` -

```

int main(int argc, char* argv[])
{
    if (argc != 3)
    {
        std::cerr << "Usage: blocking_udp_echo_client <host> <port>\n";
        return 1;
    }

    asio::io_context io_context;

    udp::socket s(io_context, udp::endpoint(udp::v4(), 0));

    udp::resolver resolver(io_context);
    auto endpoints = resolver.resolve(udp::v4(), argv[1], argv[2], std::nothrow);
    if (!endpoints)
        return report(endpoints.error());

    std::cout << "Enter message: ";
    char request[max_length];
    std::cin.getline(request, max_length);
    size_t request_length = std::strlen(request);

    auto res = s.send_to(asio::buffer(request, request_length), *endpoints->);
    if (!res)
        return report(res.error());

    char reply[max_length];
    udp::endpoint sender_endpoint;
    auto reply_length = s.receive_from(
        asio::buffer(reply, max_length), sender_endpoint, std::nothrow);

    if (!reply_length)
        return report(reply_length.error());

    std::cout << "Reply is: ";
    std::cout.write(reply, reply_length.value());
    std::cout << "\n";

    return 0;
}

```

§ References

§ Informative References

[ASIO]

Chris Kohlhoff. [Asio documentation](https://think-async.com/Asio/Documentation). URL: <https://think-async.com/Asio/Documentation>

[N4711]

Jonathan Wakely; Chris Kohlhoff. [Networking Technical Specification](https://wg21.link/n4711). URL: <https://wg21.link/n4711>

[P0323]

Vincent Botet; JF Bastien. [std::expected](https://wg21.link/p0323). April 02, 2018. URL: <https://wg21.link/p0323>